

Question 1-4 4 marks

Indicate whether the following statements are TRUE or FALSE.

1. An abstract class can ONLY contain abstract methods.
2. An interface can contain both method implementation and declaration.
3. It is a good programming practice to avoid strong coupling among classes which interact
4. Which of the following **is** or **are** true about the factory pattern?
 - A. The concrete classes can inherit a superclass
 - B. The concrete classes can implement an interface
 - C. Both A and B
 - D. None of the above

Question 5 4 marks

Class A inherits an abstract class B. In each of the following, indicate whether the statement is valid or invalid. Explain your answer.

1. B b=new B(); (2)
2. B b= new A(); (2)

Question 6 4 marks

Class C implements interface D. In each of the following, indicate whether the statement is valid or invalid. Explain your answer.

D d1=new D();
D d2=new C();

Question 7 2 marks

Give **one reason** why design patterns are important in software development.

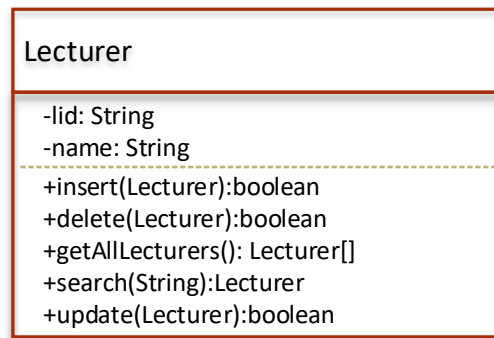
Question 8 3

Match the following examples to the main types of design patterns (3)

	Type	Example
1	Creational	A Adaptive
2	Structural	B DAO
3	Behavioural	C Factory
4		D None of the above

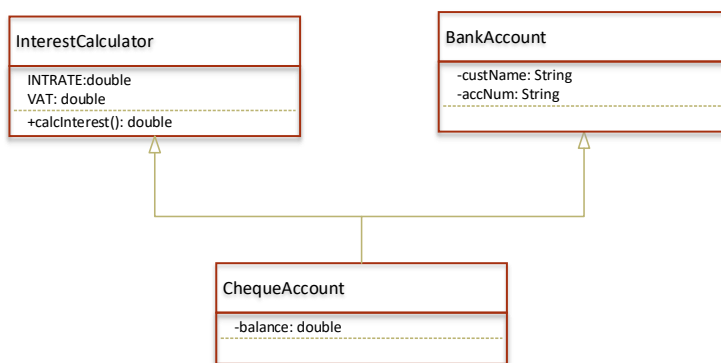
Question 9 6 marks

The diagram shows a class Lecturer. The methods **insert**, **delete**, **getAllLecturers**, **search** and **update** are operations on Lecturer objects.



1. Briefly explain the problem with the class. (2)
2. Which design pattern can resolve the problem? Briefly explain your answer. (4)

Question 10 12 marks



The diagram shows **ChequeAccount** inherits **InterestCalculator** and **BankAccount**. **BankAccount** contains **custName** (the customer name) and **accNum** (account number). **InterestCalculator** contains: **INTRATE** (interest rate at 20% per annum) and **VAT** (Vat at 15%) and **calcInterest()** returns the total interest (i.e., the 20% per annum and VAT payable) on the balance. **ChequeAccount** contains the balance (current balance).

1. Explain why **InterestCalculator** and not **BankAccount** should be converted to an interface to remove the problem of multiple inheritance. (3)
2. Write the code for the interface (**use the correct access levels**) (4)
3. Complete the code segments for 3.1 and 3.2 below (**use the correct access levels**):
 - 3.1 Complete the class header for **ChequeAccount** (after the multiple inheritance has been resolved). (3)
 - 3.2 Implement **calcInterest()**. (2)